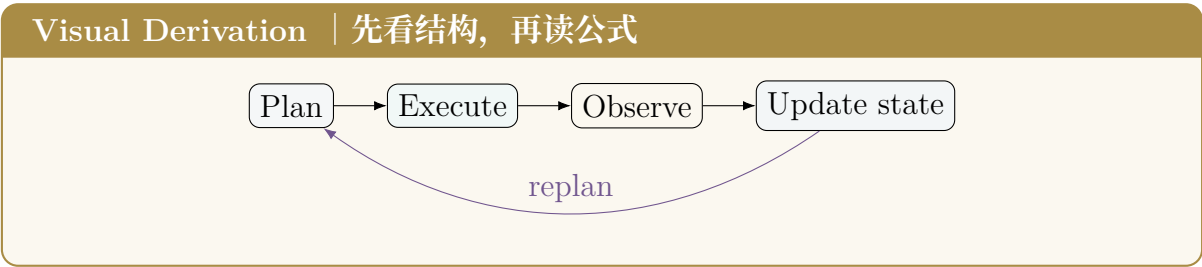


第 21 章 Tool Use 与 Planning——搜索、层级动作和闭环执行

本章问题

MDP 告诉我们“最优策略”是什么，但真实 Agent 的动作空间可能包含成百上千个工具、参数和中间计划，精确动态规划不可行。于是问题变成：**如何用有限搜索找到可执行计划，并在环境反馈推翻原计划时及时重规划？** 本章把经典图搜索、heuristic、MCTS、options 与现代 tool-calling agent 连在一起。

- Prerequisite Recall | 先把这些知识唤醒**
- 熟悉图搜索、期望回报与 stochastic transition。
 - 知道 heuristic 是对未来 cost-to-go 的估计。
 - 已掌握 POMDP 中“动作改变未来观测”的观点。



Misconception Check | 本章最容易掉进去的坑

让 LLM 生成一个很长的 plan，并不会自动得到 planning system。真正的 planning 必须与状态转移、失败观测和 replanning 结合；否则只是 open-loop sequence generation。

Concept Contrast | 把相邻概念主动分开

	Open-loop plan	Closed-loop replanning
失败后	继续原计划/崩溃	重新估计状态并规划
环境模型误差	累积	可被新观测纠正
代价	较低	需要额外 compute

Checkpoint | 开始正文前的 30 秒自测

A* 的 heuristic 若高估真实剩余成本，什么保证会丢失？**检查答案：**admissibility 被破坏，标准 A* 最优性证明不再成立；仍可能找到好解但不能沿用原保证。

21.1 Planning 与 Reasoning 的区别：动作必须改变可达状态

纯文本 reasoning 可以在内部 token 空间展开；planning 需要一个状态转移模型

$$s_{t+1} = F(s_t, a_t)$$

或随机转移 $P(s' | s, a)$ 。一个 plan 是动作序列

$$\pi_{0:H-1} = (a_0, a_1, \dots, a_{H-1})$$

使初始状态到达目标集合 $\mathcal{G}$ ，同时满足成本或约束。

在软件 Agent 中，tool call

$$a_t = (\text{tool name}, \text{arguments})$$

不是“特殊 token”，而是可执行动作。schema 决定合法动作集合，environment result 决定下一个 observation。

21.2 最短路：BFS、Dijkstra 与 A*

若确定性环境可以表示成图 $G = (V, E)$ ，边成本 $c(u, v) \geq 0$ ，则 planning 变成最短路。Dijkstra 按当前已知最小代价

$$g(n) = \text{cost}(\text{start} \rightarrow n)$$

扩展节点。A* 加入启发式

$$f(n) = g(n) + h(n),$$

其中 $h(n)$ 估计从 n 到目标的剩余成本。

定义 21.1 (Admissible heuristic). 若对所有节点 n ,

$$0 \leq h(n) \leq h^*(n),$$

其中 $h^*(n)$ 是真实最优剩余成本，则称 h 可采纳。

定理 21.1 (A* 的最优性, tree-search 版本). 当边成本非负且 h admissible 时，A* 第一次取出的 goal 节点对应最优路径。

证明思路 | 核心反证

假设 A^* 先取出一个代价 $C > C^*$ 的次优 goal。最优路径上必有尚未扩展的 frontier 节点 n 。由于 h admissible,

$$f(n) = g(n) + h(n) \leq C^* < C = f(\text{goal}),$$

所以优先队列应先取出 n , 与“先取出次优 goal”矛盾。

21.3 LLM 作为 heuristic：强大，但不保证 admissible

LLM 可以给中间状态打分、预测子目标、提出下一步工具，这很像学习到的 $h_\theta(n)$ 。但它通常既不满足 admissibility，也不满足 consistency。因此：

- LLM heuristic 可以极大减少搜索，但可能剪掉最优路径；
- verifier / simulator 可以提供更可靠的局部约束；
- “生成一个看起来合理的 plan” 与 “搜索得到可执行 plan” 必须区分。

21.4 Monte Carlo Tree Search：没有精确 heuristic 时怎么办？

MCTS 通过 rollout 估计动作价值。UCB 型选择规则常写成

$$a = \arg \max_a \left[\hat{Q}(s, a) + c \sqrt{\frac{\log N(s)}{N(s, a)}} \right].$$

第一项 exploitation，第二项 exploration。对于工具 Agent，tree node 可以是部分交互历史，edge 是工具调用，rollout reward 来自 verifier 或真实环境。

搜索越深，不代表越可靠

若 transition model、tool simulator 或 verifier 有系统误差，搜索会主动找到模型误差最大的区域。第 16 章讨论过 noisy verifier 的 winner's curse；在 planning 中，同一现象表现为 model exploitation。搜索算法越强，越需要真实环境校验。

21.5 层级规划：工具与子任务可以看成 Option

经典 hierarchical RL 定义 option

$$o = (\mathcal{I}_o, \pi_o, \beta_o),$$

分别表示启动集合、内部策略和终止条件。一个“搜索网页直到找到 DOI”“运行测试并修复直到通过”都可以看成 temporally extended action。

若 option 持续 k 步, SMDP Bellman equation 为

$$Q(s, o) = \mathbb{E} \left[\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k \max_{o'} Q(s_{t+k}, o') \right].$$

这比把每个鼠标移动或 API token 当成同一级动作更接近复杂 Agent 的真实结构。

21.6 Plan–Execute–Observe–Replan: 闭环比完美初始计划重要

如果环境随机或模型不完整, open-loop plan

$$a_{0:H-1} = \arg \max \mathbb{E}[R \mid s_0]$$

很容易在第一个意外后失效。闭环系统每执行一步就重新利用 observation:

$$h_t \xrightarrow{\text{planner}} a_t \xrightarrow{\text{tool/env}} o_{t+1} \xrightarrow{\text{state update}} h_{t+1}.$$

Model Predictive Control 的思想就是只执行当前最优计划的第一步, 然后重规划。现代 Agent 的 replanning 与它高度同构。

Research Frontier | In-the-Flow Agentic System Optimization for Effective Planning and Tool Use

会议身份: ICLR 2026 Oral。AgentFlow 把系统分成 planner、executor、verifier、generator, 并通过 evolving memory 协作; 其 Flow-GRPO 在真实多轮 loop 中优化 planner, 尝试处理长时域稀疏奖励下的 credit assignment。

教材连接: 这项工作体现了从“训练一个会输出 tool-call token 的单体模型”转向“优化闭环模块化 policy”。研究问题也从 next-token loss 转向 trajectory-level success。

官方来源: [ICLR 2026 Oral Program](#)

Research Frontier | Universal Visual Grounding for GUI Agents

会议身份: ICLR 2025 Oral。UGround 将 GUI Agent 的一个瓶颈明确分离为 visual grounding: 把语言目标映射到屏幕位置, 并探索主要依赖视觉而非 HTML/accessibility tree 的交互。

研究意义: Planning failure 不一定是“不会推理”。Action grounding 错误意味着计划正确也无法执行。Agent evaluation 必须拆开 perception、grounding、planning、execution 与 recovery。

官方来源: ICLR Proceedings

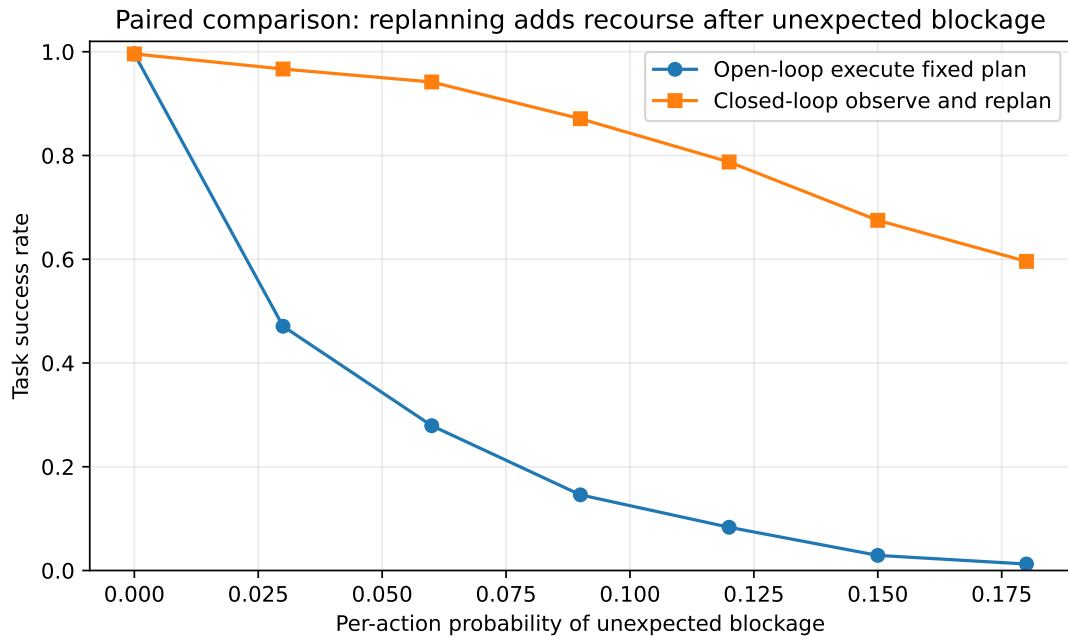


图 21.1 Frontier Lab 21: 配对使用相同静态地图与随机失败流后, 固定 open-loop plan 随意外阻塞迅速失效, 而脚本中的 observe-replan 启发式在该环境中保持更高成功率。图是机制实验; 定理 C.40 比较的是最优策略类包含关系, 不保证任意 replanning 实现都优于任意 open-loop 实现。

Mini Lab | Frontier Lab 21: A*, Tool Failure 与 Replanning

运行 `labs/frontier/ch21_planning_tools.py`。实验在随机网格中比较“规划一次后执行到底”与“每步观察后重新 A*”。请加入 tool cost、noisy observation 或 learned heuristic, 进一步测量 success、path cost、expanded nodes 与 replanning frequency。

21.7 Agent benchmark 应测什么?

只报告最终成功率会掩盖大量结构性错误。至少应区分:

plan validity, tool selection, argument correctness, grounding, recovery, cost.

不同算法可能 final score 相同, 但一个靠低成本 brute-force, 另一个靠高质量 planning; 研究结论完全不同。

From Textbook to Research | “Tool Use” 真正的下一步是什么？

当工具集合固定时，我们学习的是 policy；当 Agent 可以自己组合、生成甚至验证新工具时，动作空间本身也在变化。下一章会进一步加入 memory 与 multi-agent：Agent 不只要规划当前 episode，还要把过去轨迹转化成未来可以复用的状态与技能。

21.8 Deep Dive: Heuristic Search 与 Learned Value Function 是同一个结构

A* 的 $h(n)$ 估计 cost-to-go；RL 的 $V(s)$ 估计 future return。若把 reward 取成负成本，则

$$h^*(s) = -V^*(s)$$

只是符号不同。因此 learned value model 可以给搜索提供 heuristic，而搜索得到的新轨迹又可以反过来训练 value model。这是 AlphaZero、MCTS reasoning 与 tool planning 反复出现的闭环。

21.8.1 但价值误差会被最大化操作放大

若 $\hat{Q}(s, a) = Q(s, a) + \epsilon_a$ ，选择

$$\arg \max_a \hat{Q}(s, a)$$

会偏向正噪声动作。动作越多，极值噪声越严重。这就是 planning 版本的 optimizer's curse。于是 verifier calibration、uncertainty penalty、真实环境 rollout 都不是可有可无的工程技巧。

21.9 Deep Dive: Option 把 token horizon 缩短成 decision horizon

假设一个工具子程序平均执行 k 个 primitive steps。若高层只在 option 终止时决策，长度 H 的 primitive horizon 可以缩成约 H/k 次高层选择。这能显著缓解 credit assignment，但代价是高层策略无法干预 option 内部细节。

21.9.1 Termination 是一个学习问题

若 $\beta_o(s)$ 太小，Agent 会“做过头”；太大则 option 退化成 primitive action。现代 tool agents 的“什么时候结束搜索”“什么时候停止修代码”本质上就是 termination policy。

21.10 Deep Dive: Planning under Model Error

设 learned world model 为 $\hat{P}$ 。若每一步 transition error 的 total variation 至多 ϵ ，长 horizon 下模型 rollout 的分布误差可能累积。一个粗略直觉是有效误差尺度随 horizon 增长，因此长计划对模型准确性要求更高。

这解释了为什么真实 Agent 往往采用：短 horizon planning + frequent observation + replanning，而不是一次生成几十步后盲目执行。数学上，这是用更多真实观测减少 model bias 累积。

21.11 Deep Dive: 规划质量的六个维度

一个研究级 planner 至少可从六个维度评价：**completeness**（有解时能否找到）、**executability**（动作是否合法）、**optimality**（成本离最优多远）、**representation**（计划是否保留约束结构）、**generalization**（新工具/新环境）、**efficiency**（搜索与调用成本）。只报告 task success 会把这些维度压成一个不可诊断数字。

21.12 Proof & Worked Example Clinic

Dependency Check | 先修与跨章连接

Planning 是把第 16 章的计算分配推广到外部动作空间；失败后的 observation 又回到第 20 章 belief update。

Worked Example | A* 为什么不能只看 heuristic

A* 以 $f(n) = g(n) + h(n)$ 排序。若 h admissible，从不高估剩余代价，则定理 21.1 保证 tree-search 版本首次弹出的 goal 最优。若只按 h 排序就是 greedy best-first，可能被一个看似离目标近但累计代价巨大的路径欺骗。 g 是已付出的真实成本，不能省略。

Proof Clinic | Tool call 是带失败概率的动作

若工具 a 成功概率 p ，成功价值 V_s 、失败后可恢复价值 V_f 、调用成本 c ，则单步期望价值为

$$Q(a) = pV_s + (1 - p)V_f - c.$$

closed-loop replanning 的价值来自失败后根据 observation 重新计算 V_f ，而不是盲目重复原计划。

Research Assignment | 规划器失效注入

给工具随机注入 timeout、schema mismatch 与 stale observation，比较 open-loop plan、每步 replan、MCTS-like lookahead。除了 task success，还报告调用数、恢复率和无效循环长度。

最低交付：1 页假设与指标声明；可复现代码与固定 seed；主结果图；至少一个失败/反例；200–400 字结论边界。**禁止：**只报告最好一次 run、只给截图、把 mechanism-level reproduction 表述为 paper-scale replication。

研究阅读与复现检查表

1. 论文的 planner 是 open-loop 还是 closed-loop？何时触发 replanning？
2. 搜索中的 heuristic / verifier 是否经过校准？错误会不会被 max 操作放大？
3. tool schema 是否在测试时变化？对 unseen tools 是否真正泛化？
4. benchmark 是否区分 tool selection、argument、grounding 与 execution error？
5. 方法增加了多少 token、tool calls、wall-clock latency 与金钱成本？
6. 如果去掉 planner 只保留更强 backbone，收益还剩多少？

分层习题

- A1** 在一个 5×5 网格上手算 Dijkstra 与 A^* 的前五次 expansion。
- A2** 证明 Manhattan distance 对四邻域无负权网格是 admissible heuristic。
- A3** 给一个非 admissible heuristic 导致 A^* 返回次优解的反例。
- B1** 写出 MCTS 的 UCB selection rule，并解释 exploration constant 的作用。
- B2** 把“搜索并阅读论文直到证据充分”写成 option $(\mathcal{I}, \pi, \beta)$ 。
- B3** 推导 option 持续随机 K 步时的 SMDP Bellman target。
- C1** 设计一个 benchmark，将 tool selection error 与 argument error 分开统计。
- C2** 修改 Frontier Lab：让 observation 也有噪声，比较 blind replanning 与 belief-aware replanning。